



the route — one platform across seven phases · gates decide advancement, never the calendar

INTRODUCTION & REVIEW HANDBOOK

Twelve months. One platform. Seven gates.

A competency-gated route to working data engineering — one AWS lakehouse platform grown end to end, with retrieval practice, honest hours, and public evidence. Written to be read, used, and argued with.

Contents

Four parts, a request for your feedback, and a reference appendix.

01	The idea — what this framework is and how it works	03
02	The route — seven phases, tools, builds and gates	07
03	The projects — the platform and its nine satellites	14
04	Why it works — principles, evidence and honest costs	17
F	Help improve this framework — how to send comments	20
A	Appendix — tools, costs and where things live	21

HOW TO READ THIS HANDBOOK

Part 1 gives you the whole idea in four pages. Parts 2 and 3 are the detail — the phases, the tools, the projects. If you're here as a reviewer, everything in this document is open to challenge: page 20 tells you exactly what feedback helps most and where to send it. This handbook summarizes the repository at github.com/AlqarniMohammed/DE-projects — the repo is always the source of truth.

01

The idea

AFTER THIS PART YOU CAN

Say what a competency-gated framework is, and why gates replace deadlines.

Explain why one evolving platform beats thirty disconnected projects.

1.1

What this framework is

A twelve-month, self-paced route to working data engineering. You build **one real platform** — a full data stack, end to end: ingestion, lakehouse storage, transformation, orchestration, quality, serving — and you advance by passing **gates**: named artifacts you built and can demo, plus a terminology quiz you must score 80% on. Never by watching the calendar.

It exists because most self-study plans fail the same three ways: *tutorial hell* (watching without building), the *capstone cliff* (thirty tiny exercises, then one huge project assembled from scratch in week thirty), and *calendar schedules that cascade* the first time life happens. Every design choice in this framework traces to one of those failure modes, with the research behind it recorded in the repository.

The title says 12 months; the gates say when you're ready.
Slipping two weeks costs nothing.

Who it's for

Someone with solid SQL and basic cloud familiarity, spending 6–10 hours a week. Different starting point? The placement rule is simple: *you may start at phase N if you can pass gate N–1 today* — gates are cumulative, so two learners who hold the same gate evidence have reached the same level by construction. A full placement protocol with a worked example lives in `reference/LEARNERS.md`.

- THE STANDING RULE

Nothing is learn-only. Every resource in every phase is pinned to an exercise, a build step, or a written artifact — because unexercised knowledge evaporates. If a row in a Learn table has no exercise attached, that's a bug worth reporting.

1.2

How a phase works

Every phase has the same shape, and you repeat it seven times. **Words first:** each phase opens with its new glossary terms, taught as contrast pairs (OLTP vs OLAP, SCD1 vs SCD2) because contrast sticks. Then the Learn table, one row at a time — every row names a resource *and the exercise that pins it*. Several rows offer a vetted video twin: you pick one lane before starting the row, and hours swap — they never add.

Then you build: grow the platform through pull requests (branch → PR → self-review → squash-merge, even working solo), do the

phase's **prove-it assignment** — a small artifact that makes the phase's hardest claim concrete — and build the phase's satellite project in its own repo.

Finally, the gate. A gate is a fixed checklist: the named artifacts, a **retrieval checkpoint** (all current-phase terms plus 10 random earlier ones, $\geq 80\%$ — spaced retrieval keeps old phases warm), from G2 onward a **requested external critique** — the framework never grades itself alone — and one public post. Evidence links go in a tracker; an item counts only when proof is linked.

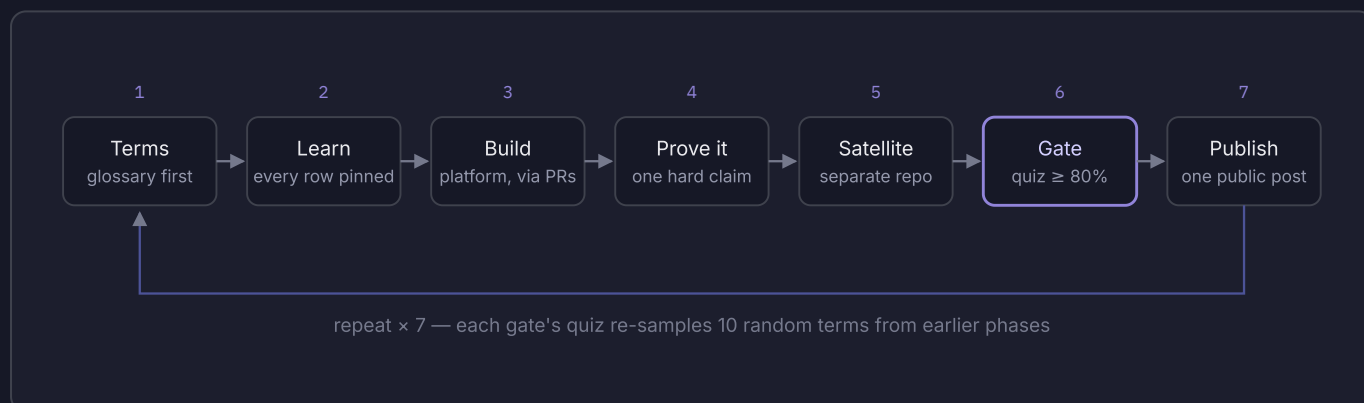


Fig. 1.1 The phase loop. The gate step is the only exit from a phase; six built-in Claude Code skills (`/start-phase`, `/quiz`, `/gate-check`, `/satellite-brief`, `/explain`, `/resume`) automate the repetitive parts, and every flow has a manual path.

1.3

The route at a glance

Seven phases, one growing platform. The month columns are a pace suggestion, not a contract — phases 2 and 3 say "elastic, the gate decides" in their own headers.

PHASE	MONTHS	HOURS	YOU BUILD	GATE
0 · Orientation & Setup	~2 weeks	18	Toolchain + guard-railed AWS account	G0
1 · Foundations	1–2	70	A lakehouse on your laptop (DuckDB · Iceberg · dbt)	G1
2 · AWS Lakehouse Core	3–5	95	The same lakehouse on AWS (S3 · Glue · Athena · S3 Tables)	G2 · AWS DEA-C01
3 · Spark & Databricks	5–7	75	Spark fluency + seven Databricks labs	G3 · Databricks DE
4 · Orchestration & Ingestion	7–8	72	Airflow 3, dlt, CDC and Kinesis on the platform	G4
5 · Streaming & CDC	9–10	65	Kafka event path → lakehouse; Debezium vs DMS	G5
6 · Production, Serving & Capstone	11–12	88	Quality, lineage, Terraform, serving, AI pipeline	G6 · capstone

~483 h

the honest total — every phase budget published, practice-exam time counted

≤ \$25/mo

the AWS ceiling all year, protected by a budget alarm you force-fire on day one

2 certs

AWS DEA-C01 and Databricks DE Associate — both passed by ≈ month 7

02

The route: seven phases

AFTER THIS PART YOU CAN

Name each phase's tools and what its core build adds to the platform.

Say what each gate certifies — the bar stays fixed even when scenarios vary.

2.1 Phase 0 — Orientation & Setup

~2 weeks · ~18 h · AWS \$0

Only setup, on purpose — the most-reported failure of public DE courses is the week-1 setup wall, where Docker, cloud and new concepts land at once. You install the toolchain (uv, Docker, VS Code, Git, the DuckDB CLI, a gitleaks pre-commit hook), create a guard-railed AWS account — a \$25/month budget whose alarm you *force-fire once*, because an alarm you've never seen fire is a hope, not a guardrail — and run your first DuckDB query over a Parquet file of NYC taxi trips.

Prove it: the Anki generator — a hand-typed Python script that parses the glossary (its grammar is strictly machine-parseable) into a flashcard deck, with a pytest that fails loudly on a malformed line. It powers every retrieval checkpoint of the year.

GATE G0 CERTIFIES — you can speak the DE lifecycle and its core contrast pairs, and operate a guard-railed cloud account and local toolchain.

2.2 Phase 1 — Foundations

months 1-2 · ~70 h · AWS \$0 — everything runs on your laptop

A lakehouse on your laptop, one dataset carried end to end. Platform v0.1: an idempotent taxi-data ingester (typed, tested), the data registered as an **Apache Iceberg** table and deliberately exercised — schema evolution, snapshots, time travel — then a three-layer **dbt** project (staging → intermediate → marts) with 25+ tests, generated docs, and a snapshot that makes `dim_zones` a documented SCD2 dimension. `make demo` runs the whole thing on a fresh clone. The PR workflow starts here and never stops.

TOOLS — analytical SQL (DataLemur / LeetCode SQL 50) · Python + pytest · DuckDB · Polars · Arrow · Iceberg (Pylceberg) · dbt. SATELLITE S1 — a standalone dbt project from zero, in its own repo, so you start a dbt project twice: once guided, once alone.

GATE G1 CERTIFIES — interview-strength analytical SQL; tested DE Python; a table format operated (schema evolution, time travel); a documented 3-layer dbt project with an SCD2 dimension.

2.3

Phase 2 — AWS Lakehouse Core

CERT PHASE 1

months 3–5 (elastic) · ~95 h · AWS $\leq$ \$25/mo + exam \$150 + practice exams \$15

Your lakehouse moves to AWS. Same project, real cloud — and the services you operate are exactly the surface the **AWS Certified Data Engineer – Associate** exam tests, so the build *is* most of the prep. Raw and curated S3 zones; a Glue crawler you then replace with explicit DDL (and learn why teams do); the curated layer written as managed Iceberg on **S3 Tables** by a hand-written ~40-line Glue PySpark job; the same dbt project re-pointed at **Athena**, with an incremental fact model and a `make backfill` idempotency proof — run it twice, diff the row counts.

The phase's signature move is **cost telemetry**: a script that prints scanned bytes per dbt model, one CTAS and one partition-projection drill, and a screenshot of the same query before and after partitioning — a 10–100× scan reduction you can show, not claim. Two short exam mini-labs (Kinesis → Firehose, Redshift Serverless taste) are built and torn down the same day.

Prove it: pipeline tests with `pytest + moto` (mocked S3), wired into GitHub Actions via OIDC so a red test blocks the nightly run. CI is not decoration — it's the thing that stops a broken pipeline from shipping.

TOOLS — S3 · Glue (Catalog, Crawlers, ETL) · Athena · S3 Tables · dbt-athena · Lake Formation · PySpark basics · `pytest + moto` · GitHub Actions OIDC. **SATELLITE S2** — a cost-aware Athena mart on a fresh 5–10 GB dataset where *every model is tagged with its dollar cost per build*, published as a `MODEL_COST_REPORT.md` leaderboard.

GATE G2 CERTIFIES — a production-pattern lakehouse on AWS; idempotent backfills proven; query costs explained and cut; pipelines tested in CI; the monthly bill $\leq$ \$25 with evidence; **AWS DEA-C01 passed**. From this gate on, every gate also requires requesting an external critique from a practitioner community.

2.4

Phase 3 — Spark & Databricks

CERT PHASE 2

months 5–7 (elastic) · ~75 h · platform \$0 · exam \$200 → \$100 with voucher

Distributed compute enters. The exam labs run on **Databricks Free Edition** — serverless, Unity Catalog on by default, exactly the environment the current exam assumes — while the Spark-internals work runs on local PySpark, where the full Spark UI is available. Seven labs map one-to-one to the exam sections: ingestion (Auto Loader, COPY INTO), a PySpark medallion build, Delta Lake mechanics (MERGE, the transaction log, Liquid Clustering), a Lakeflow Jobs multi-task DAG with file-arrival and table-update triggers, a declarative pipeline with expectations, Unity Catalog governance (grants, a column mask, a row filter), and a CI/CD Asset Bundle deployed dev → prod.

Prove it: the skew-detection kata — a hand-written `detect_skew(df, key)` helper with your own `pytest` asserts against a uniform and a 90/10-skewed `DataFrame`. It becomes the measuring instrument for the forensics satellite.

TOOLS — PySpark / Spark 4.x · the Spark UI · Delta Lake · Databricks Free Edition · Unity Catalog · Lakeflow (Connect, Jobs, Spark Declarative Pipelines) · Asset Bundles + the Databricks CLI · Pylceberg & delta-rs for the bake-off. **SATELLITES** — **S3a Spark Performance Forensics:** a deliberately slow job fixed one change at a time (broadcast, AQE, salting, repartitioning, file sizing), published as `FORENSICS.md` with before/after Spark-UI screenshots and a ranked impact table. **S3b Iceberg vs Delta bake-off:** the same workload through Pylceberg and delta-rs, benchmarked — write, read, schema evolution, time travel — published as `BAKEOFF.md` with real numbers and a verdict.

GATE G3 CERTIFIES — PySpark at Spark-UI-literacy level; skew explained *and fixed*; Delta, Lakeflow and Unity Catalog operated; **Databricks Certified Data Engineer Associate passed.**

2.5

Phase 4 — Orchestration & Ingestion

months 7-8 · ~72 h · AWS ≤ \$25/mo with per-lab caps and teardown deadlines

Deliberately exam-free: Kinesis, DMS and Redshift arrive as services you've already answered exam questions about, now operated for real with no prep clock running. Platform v2 gets a real orchestrator — **Airflow 3**, run locally, as *many small single-purpose DAGs, never a mega-DAG*: retries, a Deadline Alert (Airflow 3's replacement for the removed SLA feature), failure alerts, one asset-triggered dependency, one deliberate XCom handoff.

Then three ingestion modes join the batch path: a declarative **dlt** REST-API source with an incremental cursor; **CDC** from a live RDS Postgres through DMS into Iceberg — capped at \$8, torn down within five days, with an UPDATE in Postgres visible in Athena in minutes; and a streaming-lite EventBridge → Lambda → **Kinesis** → Firehose path capped at \$5. The secrets story also becomes production-shaped: every credential moves to the SSM Parameter Store backend — none in code, config files, or git.

Prove it: DAG-integrity tests in CI — every DAG imports cleanly, has retries and an owner, no orphaned tasks, the XCom handoff carries the expected value. A broken DAG never reaches the scheduler.

TOOLS — Airflow 3.x + Astro CLI · dlt · SSM Parameter Store · RDS Postgres + DMS · Kinesis Streams & Firehose · EventBridge, Lambda, Step Functions · MWAA (one short lab) · Dagster. **SATELLITES** — **S4a Redshift + Spectrum hybrid**: a mart joining native and Spectrum tables, with the federated EXPLAIN plan as proof. **S4b Dagster vs Airflow**: one platform pipeline rebuilt as software-defined assets, compared in **COMPARISON.md** across four fixed axes with a code diff each.

GATE G4 CERTIFIES — orchestration with a deliberate failure → retry → alert shown on video; ingestion via dlt, CDC and Kinesis; secrets handled like production, with gitleaks shown catching a planted secret.

2.6

Phase 5 — Streaming & CDC

months 9–10 · ~65 h · near-\$0 — Kafka runs locally in Docker

Open-source streaming, because Kafka's mechanics transfer everywhere — you already ingested streams the AWS way in Phase 4. Platform v3 grows the event path: a local **Kafka 4.x / Redpanda** broker; a producer replaying real events; two consumer groups doing different jobs, with consumers killed and restarted mid-stream so partition ownership and offsets are felt, not memorized. The sink is the clever part: a local consumer writing to the *cloud-side* curated table via **Pylceberg** against the **Glue/S3 Tables REST** catalog — local compute, cloud table — which is what makes "an event is visible in Athena within minutes" a real demo.

Prove it — the idempotent sink: first *prove the failure* — send duplicates through a naive Iceberg append and watch them land twice — then fix it with `upsert()` on the event key, resend the duplicates, and show the table stays correct. At-least-once delivery plus an idempotent sink is effectively exactly-once; this is the phase's hardest claim made concrete, on video.

TOOLS — Kafka 4.x KRaft / Redpanda · confluent-kafka (Python) · Kafka Connect · Debezium · Pylceberg upsert · one Flink SQL lab (windows and watermarks, semantics only) · the free Confluent *Data Streaming Engineer Foundations* certificate. **SATELLITE S5 — Debezium vs DMS, the CDC showdown:** identical workloads (the seeded OLTP generator built in Phase 4) through both paths, published as **CDC-SHOWDOWN.md** — a measured latency histogram, schema-change and delete handling in both, the outbox pattern demonstrated, and a verdict. Nobody publishes honest side-by-side CDC numbers; after this, you have them.

GATE G5 CERTIFIES — you can reason about partitions, offsets and delivery guarantees from experience; build an idempotent stream-to-lakehouse sink; and compare CDC approaches with your own measurements.

2.7

Phase 6 — Production, Serving & Capstone

months 11-12 · ~88 h · AWS $\leq$ \$25/mo — quality, lineage and serving run locally or inside dbt

The year's final move: make the platform read like production. Platform v4 layers **quality** (dbt tests → dbt-expectations distributional tests → Elementary anomaly detection with alerts, plus a quarantine pattern built as its own task), enforces **contracts** on the two most-consumed marts with a CI check that fails a breaking PR, emits **OpenLineage** events from Airflow into Marquez for a cross-DAG lineage graph, codifies the core infrastructure in **Terraform** (imported, clean plan, tfsec-scanned), and writes the runbook.

Prove it — the contract that blocks a PR: open a deliberately breaking PR (rename a contracted column), let CI reject it, and leave the blocked PR open as permanent evidence. The strongest "I run this like production" proof there is.

TOOLS — dbt-expectations + Elementary · dbt model contracts · OpenLineage → Marquez · Terraform + tfsec · CloudWatch · ClickHouse · Superset · pgvector · DDIA 2E as the theory backstop. **SATELLITES** — **S6a ClickHouse + Superset serving layer:** two gold marts replicated on a schedule (the hot-copy pattern; the lakehouse stays the source of truth), a customer-facing dashboard, and a *measured* latency number vs Athena. **S6b the AI-data pipeline:** documents → chunks → embeddings → pgvector with idempotent upserts and a retrieval-quality check as the pipeline's "dbt test" — a normal data pipeline with a new sink.

The capstone is packaging, not a new build — the platform was built all year instead of assembled in week 30. Self-grade against a seven-axis rubric, rewrite the README for a five-minute stranger read, record three ≤ 3 -minute demos, and write JOURNEY.md — the platform-journey article — plus AI-USAGE.md, an honest record of what you delegated to AI versus hand-typed.

GATE G6 CERTIFIES — the whole platform, explained end to end: rubric ≥ 3 average, a caught anomaly, the blocked PR, the lineage graph, a serving-latency number, a clean terraform plan — and a final retrieval checkpoint sampling 30 terms across all phases.

03

The projects

AFTER THIS PART YOU CAN

Describe the platform's v0.1 → v4 arc and what each phase added to it.

Pick the satellite whose published artifact you'd most want to review.

3.1

One platform, grown all year

The centerpiece is a single repository, `aws-lakehouse-platform`, created in Phase 0 and grown for twelve months. `v0.1` is a lakehouse on a laptop. `v1` moves it to AWS. `v2` gives it an orchestrator and three more ingestion modes. `v3` adds the event path. `v4` hardens it: quality, contracts, lineage, laC, a serving layer.

One platform is the framework's central bet. It avoids the capstone cliff, and it changes what the repo *is*: a year of pull-request history with tests, teardowns and cost screenshots reads

as an operated platform, not a tutorial. Every claim gets a fixed, named artifact —

`MODELING.md`, `PLATFORM-RUNBOOK.md`, `JOURNEY.md` — because work never gets credit for an invisible strength.

The destination is the kind of full stack real data teams publish journey posts about — and the capstone write-up, `JOURNEY.md`, tells yours: the problem, the architecture, the decisions with reasons, the numbers, the lessons.

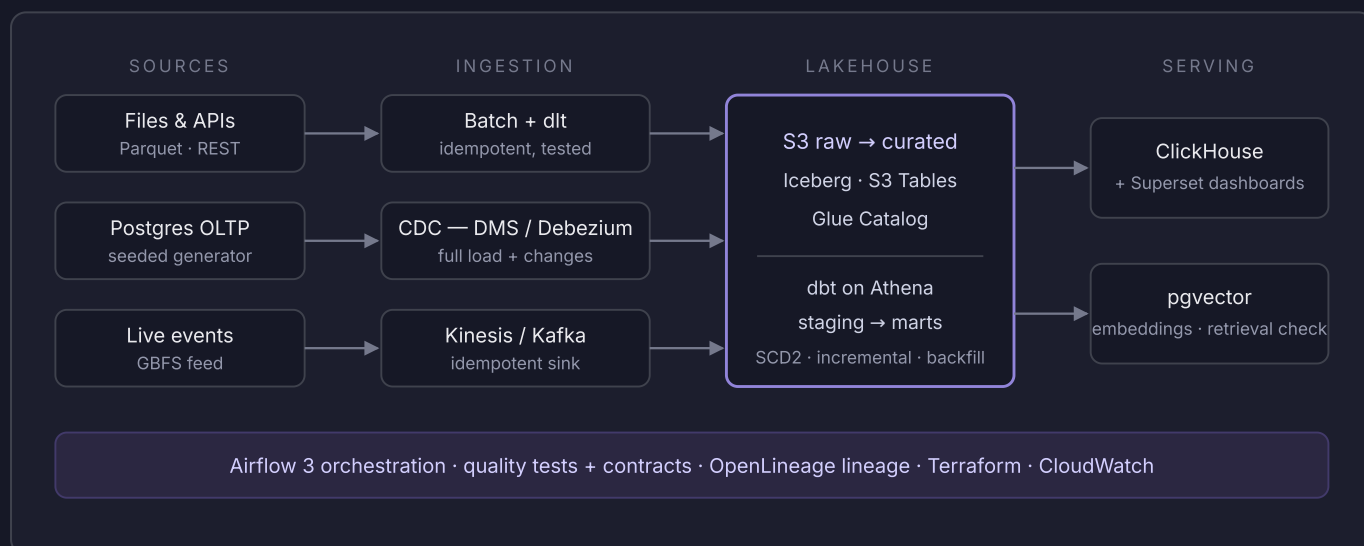


Fig. 3.1 Platform v4. Batch, API, CDC and stream ingestion into an Iceberg lakehouse on S3; dbt on Athena for transformation; a ClickHouse hot copy and a pqvector sink for serving; orchestration, quality and lineage cut across every layer.

3.2

The nine satellites

Separate repos, one or two per phase, for tool diversity the platform's default stack can't give. Each ends in a published, named artifact. The default specs are complete; a built-in generator can also issue a unique, realistic business brief for any of them — with black-box acceptance tests and a sealed reference solution — while the objectives and the gate bar stay fixed.

ID	SATELLITE	PHASE	THE PUBLISHED ARTIFACT
S1	Your first standalone dbt project	P1	A live docs site — 20+ models, 40+ tests, one custom generic test
S2	Cost-aware Athena mart	P2	MODEL_COST_REPORT.md — dollar cost per model, one fix halving the top cost
S3a	Spark performance forensics	P3	FORENSICS.md — before/after Spark-UI evidence, fixes ranked by impact
S3b	Iceberg vs Delta bake-off	P3	BAKEOFF.md — the same workload through both formats, measured, with a verdict
S4a	Redshift + Spectrum hybrid	P4	A federated EXPLAIN plan + a cost note vs all-Redshift
S4b	Dagster vs Airflow	P4	COMPARISON.md — four fixed axes, one code diff each
S5	Debezium vs DMS — the CDC showdown	P5	CDC-SHOWDOWN.md — measured latency histogram, tombstones, the outbox pattern
S6a	ClickHouse + Superset serving layer	P6	A customer-facing dashboard + the measured latency delta vs Athena
S6b	The AI-data pipeline	P6	An embeddings → pgvector pipeline with a green retrieval-quality check

● WHY THESE NINE

Each satellite either forces a second start-from-zero (S1), produces a comparison the internet is missing (S3b, S4b, S5), or makes an invisible skill visible with a number (S2's dollar costs, S3a's ranked fixes, S6a's latency delta). If one doesn't earn its hours, that's exactly the feedback page 20 asks for.

04

Why it works

AFTER THIS PART YOU CAN

Trace each design decision to the failure mode it prevents and the evidence behind it.

Say what a finisher holds at the end — and what the whole year costs.

4.1

The principles and the evidence

The framework was compiled from six parallel deep-research reports — tool landscape, cert blueprints, courses, real platform case studies, roadmap benchmarks, market signals — then independently audited twice and revised three times. Nine principles carry the design:

- 1 **SQL and Python are the permanent spine** — in 70–94% of postings; exercised in every phase.
- 2 **Idempotency + retries is the reliability model** — the highest lesson-density skill across every platform case study.
- 3 **Many small jobs, never a mega-DAG** — two teams independently regretted the monolith within a year.
- 4 **Three modeling layers from day one** — two teams independently regretted two.
- 5 **Table maintenance is the lakehouse's hidden cost** — compaction and snapshot expiry are first-class objectives.
- 6 **Concepts are the skeleton; tools are replaceable leaves** — when a tool churns, the phase objective survives.
- 7 **Builds must survive weeks of inattention** — serverless over self-hosted; the framework never asks you to run Kubernetes.
- 8 **Nothing is learn-only** — every resource is pinned to an exercise; unexercised knowledge evaporates.
- 9 **Ship the phase, then improve it** — a gate passed at "good" beats a phase polished forever.

Spaced retrieval is the framework's original contribution — none of the nine surveyed DE roadmaps has it. Certs sit in the first half because each exam's surface is exactly what its phase builds.

- KEPT HONEST, STRUCTURALLY

External critique is a gate requirement from G2 on — solo learners systematically miss their own gaps, so the framework never grades itself alone. Contested calls (Iceberg as CORE, Airflow over Dagster, Debezium's place) are listed as judgment calls in the open, and each phase starts with a "re-verify before starting" check because tool versions and exam blueprints churn.

4.2

What you finish with, and what it costs

The definition of done: all seven gates passed with linked evidence. In practice you can design, build, operate and explain a batch + streaming lakehouse end to end; prove your data is correct; and say what it costs to run.

THE FINISHER'S INVENTORY	EVIDENCE
The platform, v4	One repo, a year of PR history, three demo videos, a runbook
Nine satellite repos	Each with a published, named artifact — several with numbers nobody else publishes
Two certifications	AWS DEA-C01 · Databricks DE Associate (+ a free Confluent streaming certificate)
The write-ups	JOURNEY.md · AI-USAGE.md · SELF-CHECK.md — a year of written answers
A public record	~9 posts, one per phase — the year's arc from local lakehouse to journey post

The honest hour math. Phase budgets sum to ~483 hours (18 + 70 + 95 + 75 + 72 + 65 + 88). At 9–10 h/week that's the 12-month arc; at 8 h/week, ~15 months; at 6 h/week, ~20 months — same destination, later arrival. Two buffer weeks per half-year are planned in. If time runs short there is a pre-authorized trim path — S4a first, then the Polars and Flink labs, then S1 — and trimming an item also strikes it from the gate, so the evidence trail stays honest.

\$310–455

the whole year's exam-and-course spend; the low end assumes the Databricks voucher

12–20 mo

the honest range by weekly pace — gates don't care about calendars

0 trims

ever allowed on the platform or the two certs — everything else has a stated order

F

Help improve this framework

This handbook exists to collect criticism. The framework was built from research and refined through three audit-driven revisions — but it has not yet been walked end to end by many learners, and that is the gap your comments close. External feedback is already a structural requirement inside the framework; this page applies the same rule to the framework itself.

What we most want to know

- 01 **Hours.** Are the phase budgets realistic at your pace — where did the estimate break?
- 02 **Tools.** Any CORE / AWARE / SKIP verdict you'd overturn — and the evidence for it?
- 03 **Gates.** Is any bar too high, too low, or impossible to verify honestly?
- 04 **Satellites.** Which one earns its hours, which one doesn't, and what's missing?
- 05 **Gaps.** What would you add — and, just as important, what would you cut to make room?
- 06 **Clarity.** Where did the documents lose you? "Step 4 lost me" is a first-class bug report.

Known gaps, stated rather than hidden: some tool weightings derive from one regional market's signals and should be re-derived for other markets; two course platforms (DataCamp, Udacity) were never researched; the full list of evidence gaps lives in `reference/SOURCES.md`.

• WHERE TO SEND IT

Open one GitHub issue per point at github.com/AlqarniMohammed/DE-projects/issues — quote the page number from this handbook or the file in the repo. Small fixes (typos, dead links) are welcome directly as pull requests. Every issue gets read; decisions and their reasons land in the changelog.

Tools, terms and where things live

Condensed from the repository's reference shelf. Every tool below links to its official docs there — when a tutorial and the docs disagree, the docs win.

CORE — hands-on, scheduled in a phase

SQL · Python (uv, pytest, moto) · Parquet · DuckDB · Apache Iceberg · dbt · S3 · Glue · Athena · S3 Tables · Lake Formation · Lambda · PySpark · Delta Lake · Databricks · Airflow 3 · dlt · DMS · Kinesis · Redshift · SSM + gitleaks · Kafka · Debezium · dbt-expectations + Elementary · Terraform · Superset · ClickHouse · pgvector · Docker · Git + GitHub Actions.

AWARE — concepts + at most a day

Polars · Flink · Redpanda · MSK · MWAA · EMR · Step Functions + EventBridge · Dagster · SQLMesh · UniForm · DuckLake · Polaris · DataHub / OpenMetadata · OpenLineage / Marquez · Soda · Great Expectations · Monte Carlo · semantic layers · Airbyte / Fivetran / Sling · Snowflake · the Azure vocabulary. Rule: every AWARE item ends in one written comparison sentence — awareness that produced no sentence didn't happen.

Further reading

1. The repository — github.com/AlqarniMohammed/DE-projects (CC-BY-4.0). Start with README, then GUIDE.md.
2. [reference/WHY.md](#) — every design decision with the evidence behind it, including the hour math and the trim rules.
3. [sources/research/](#) — the six deep-research reports the framework is compiled from.

SKIP — deliberately excluded, with reasons

Hadoop/HDFS ops · Apache Hudi · Prefect and Kestra · StarRocks/Doris · Kubernetes administration · Scala · deep Flink · a third cloud · a third certificate. Each exclusion is argued, not assumed — and each is fair game for feedback.

Where things live in the repo

README the front door · GUIDE.md the operating manual · GLOSSARY.md 98 terms, machine-parseable · PROGRESS.md the evidence tracker · phases/ the seven specs · [reference/](#) WHY, TOOLS, CERTS, COURSES, DATASETS, LEARNERS, SOURCES · prompts/ the satellite-brief generator · [.claude/skills/](#) six helper skills.



Start at Phase 0. Follow the loop. The gates tell you when you're done — and the issues tab is open for everything you'd change.